Philipp Schäfer
Tragwerkslabor
Fachbereich Technik | Engineering

Hochschule Mainz
University of Applied Sciences
Holzstraße 36, 55116 Mainz

T +49 6131 628-1366
philipp.schaefer@hs-mainz.de

IFC → SCENE GRAPH → JAXFEM

Technische Dokumentation | Version 1.0

Vollautomatische FEM-Berechnung aus IFC-Gebäudemodellen über semantische Scene Graphs, DBSCAN-Clustering und den JAX-basierten Solver JaxFEM

Mainz, April 2026

Zusammenfassung

Das Jupyter-Notebook `IFC_SceneGraph_FEM.ipynb` realisiert eine vollautomatische Pipeline von einem IFC-Gebäudemodell [12] über einen semantischen Scene Graph (NetworkX `DiGraph`) bis zu einer Finite-Elemente-Berechnung mit dem differenzierbaren JAX-Solver JaxFEM [11]. Alle Phasen der Modellvorbereitung — Bauteil-Parsing, DBSCAN-Clustering, Beziehungsanalyse, Randbedingungsableitung, Materialzuweisung und Netzgenerierung — erfolgen ohne manuelle Eingriffe. Die Ergebnisse (nodale Verschiebungen, Von-Mises-Spannungen) werden in den Scene Graph zurückgeschrieben und als strukturierter LLM-Prompt für Spatial-AI-Abfragen exportiert. Der erste vollständige Durchlauf auf der öffentlichen Beispieldatei `Ifc2x3_Duplex_Architecture.ifc` mit 149 IFC-Bauteilen, 13 Graphknoten, 1 958 Tet4-Elementen und 1 917 Freiheitsgraden war erfolgreich abgeschlossen. Das Dokument beschreibt jeden der 20 Notebook-Schritte in der Reihenfolge ihrer Ausführung — von den Designentscheidungen und der Algorithmuswahl bis zu den konkreten Parametern, Datenstrukturen und Ergebnisinterpretationen.

0 INHALTSVERZEICHNIS

1 Motivation und Einordnung **3**
1.1 Ausgangsproblem: die BIM-to-FEM-Lücke . . . 3
1.2 Wissensrepräsentation als Schlüssel . . . 3
1.3 Differenzierbarkeit und JAX . . . 3
1.4 Einordnung in die Projekthistorie . . . 3

2 Gesamtarchitektur der Pipeline **4**

3 Konfigurationsparameter (Schritt 1) **4**

4 Phase 1: Modellaufbau **6**
4.1 IFC laden und Demo-Modell (Schritt 2) . . . 6
4.1.1 Demo-Modell . . . 6

4.1.2 Eigene IFC-Datei und Z-Normalisierung . . . 6
4.2 Bauteil-Parsing und Punktwolkenerzeugung (Schritt 3) . . . 6
4.3 IFC-Relationship-Analyse (Schritt 5) . . . 6
4.4 Semantisches DBSCAN-Clustering (Schritt 6) . . . 7
4.4.1 Clustering-Algorithmus . . . 7
4.4.2 IFC-BBox-Snap . . . 7
4.5 Feature-Extraktion (Schritt 7) . . . 7
4.6 Räumliche Beziehungsanalyse (Schritt 8) . . . 7

5 Scene Graph — Formale Beschreibung (Schritt 9) 7
5.1 Graphstruktur . . . 7
5.2 Graph-Metriken und Analysefunktionen (Schritt 10–11) . . . 8
5.3 Automatische Ableitung der FEM-Randbedingungen . . . 8

6 Phase 2: FEM-Vorbereitung 9
6.1 Material-Mapping (Schritt 12) . . . 9
6.2 GMSH Tet4-Netz aus IFC-Geometrie (Schritt 13) . . . 9
6.2.1 Strategie: STL-Merge . . . 9
6.2.2 Fallback-Strategie . . . 9
6.2.3 Netzausgabe . . . 9
6.3 Randbedingungen (Schritt 14) . . . 10
6.3.1 Kontaktzonenbasierte BCs . . . 10
6.3.2 JAX-kompatible Location-Funktionen . . . 10

7 FEM-Formulierung (Schritt 15–16) 10
7.1 Konstitutives Gesetz — Lineare 3D-Elastizität . . . 10
7.2 Schwache Form und FEM-Diskretisierung . . . 10
7.3 Tet4-Element . . . 11
7.4 JaxFEM-Problemklasse . . . 11
7.5 Von-Mises-Vergleichsspannung und Spannungsberechnung (Schritt 16) . . . 11

8 Ergebnisse — Erster Meilenstein (Schritt 17–18) 12
8.1 Modell-Kenngrößen . . . 12
8.2 FEM-Ergebnisse . . . 12
8.3 Plausibilitätsbewertung . . . 12
8.4 Ergebnisrückfluss in den Scene Graph (Schritt 18) . . . 13

9 LLM-Integration (Schritt 19) 13
9.1 Prompt-Generierung . . . 13
9.2 Anwendungsbeispiele . . . 13
9.3 API-Anbindung . . . 13

10 Vergleich: Vorgänger vs. neues Notebook 14

11 Bekannte Einschränkungen (Version 1.0) 14

12 Roadmap und Nächste Entwicklungsschritte 15

13 Softwareabhängigkeiten und Umgebung 15
13.1 Python-Pakete . . . 15
13.2 Conda-Umgebung . . . 16
13.3 Ausgabedateien . . . 16

14 Zusammenfassung aller Notebook-Schritte 17

Referenzen 17

1 MOTIVATION UND EINORDNUNG

1.1 Ausgangsproblem: die BIM-to-FEM-Lücke

Die Vorbereitung eines FEM-Modells aus einem BIM-Gebäudemodell ist in der ingenieurpraktischen Anwendung bis heute ein manuell geprägter Prozess. Der typische Workflow umfasst den Export der Geometrie aus dem BIM-Authoring-Tool (Revit, ArchiCAD, Vectorworks), den Import in ein Analyse-Programm (RFEM, ANSYS, Abaqus, SOFiSTiK), die manuelle Definition aller Randbedingungen und Lastflächen, die Materialzuweisung und schließlich die Netzerstellung — ein Prozess, der bei komplexen Modellen mehrere Personentage in Anspruch nehmen kann.

Diese Lücke ist ein seit Jahren dokumentiertes Forschungsproblem. Jin et al. [4] identifizieren in ihrer bibliometrischen Analyse von über 900 Publikationen Interoperabilität, die fehlende Semantik des geometrischen Datenmodells und die Diskrepanz zwischen „As-designed und „As-builtLeistung als die drei zentralen Hemmnisse. Die vorliegende Pipeline adressiert alle drei Punkte: IFC liefert das semantische Vokabular, der Scene Graph schließt die Interoperabilitätslücke durch eine topologiebasierte Inferenz, und die offene Architektur erlaubt künftig die Einspeisung von Scan-to-BIM-Daten [9, 10].

1.2 Wissensrepräsentation als Schlüssel

Der Kern der Neuerung gegenüber früheren BIM-to-FEM-Ansätzen liegt in der expliziten Wissensrepräsentation als gerichteter Graph. Anstatt die IFC-Geometrie direkt in ein Netz zu überführen, wird zunächst ein semantischer Scene Graph $G = (V, E)$ aufgebaut, der alle Bauteilinstanzen als Knoten und ihre räumlichen sowie ontologischen Beziehungen als Kanten enthält. Erst dieser Graph liefert die Information, die für eine automatische Ableitung der Randbedingungen und Lastflächen benötigt wird: Welche Wände stehen frei (Einspannung am Fuß)? Welche Decke liegt oben (Lastauflagefläche)?

Poux & Lehtola [1] zeigen in ihrer Arbeit zu 3D Scene Graphs für Spatial AI, dass diese Repräsentation für LLM-gestützte Abfragen besonders geeignet ist, da sie gleichzeitig geometrische und semantische Information in einer für Sprachmodelle verständlichen, strukturierten Textform serialisierbar macht. Wang et al. [7] demonstrieren analoge Ideen für die automatische Generierung von Gebäudeenergiemodellen aus BIM-Daten über wissensgraphbasierte Frameworks; eine Folgearbeit [8] zeigt die Robustheit solcher Ansätze auch bei unvollständigen IFC-Daten. Alves et al. [5] kartieren 14 BIM- und 16 KI-Kompetenzen als Grundlage smarter Bauprojekte und bestätigen den Trend zur semantischen Anreicherung. Peng et al. [6] zeigen, wie Wissensgraphen im Facility-Management-Bereich eingesetzt werden, um strukturell ähnliche Fragen über Bauteilbeziehungen zu beantworten.

1.3 Differenzierbarkeit und JAX

Ein weiterer Vorteil der Wahl von JaxFEM [11] als Solver liegt in der JAX-basierten Implementierung: Der gesamte FEM-Solver ist durch `jax.grad` differenzierbar. Das öffnet den Weg zu gradientenbasierter Topologieoptimierung und Formoptimierung direkt aus dem Notebook heraus — ohne Umbau der Solver-Architektur. Dies ist Prio 7 der Roadmap (Abschnitt 12).

1.4 Einordnung in die Projekthistorie

Tabelle 1: Notebook-Generationen des JaxFEM-Projekts

Notebook	Netzgeometrie	Neuerung
`kragarm.ipynb`	Analytisch	Erster JaxFEM-Test
`Rechteckbalken_3D.ipynb`	Manuell (Quader)	3D-Elastizität
`IFC_FEM_3D.ipynb`	BBox-Tet4	Erster IFC-Import
`SceneGraph_FEM.ipynb`	BBox-Tet4	Synthetischer Scene Graph
`IFC_SceneGraph_FEM.ipynb`	Echte IFC-Geometrie	**Vollautomatische Pipeline**

2 GESAMTARCHITEKTUR DER PIPELINE

Die Pipeline gliedert sich in 20 sequenzielle Notebook-Schritte, die in drei logische Phasen eingeteilt werden können:

Phasenübersicht

Phase 1 — Modellaufbau (Schritte 0–9): IFC-Datei laden, Bauteile parsen, Punktwolken erzeugen, IFC-Relationships extrahieren, DBSCAN-Clustering, Feature-Extraktion, räumliche Beziehungsanalyse, Scene Graph aufbauen.

Phase 2 — FEM-Vorbereitung (Schritte 10–14): Graphanalyse für Randbedingungen, Material-Mapping, GMSH Tet4-Netz, Randbedingungs-Kontaktzonen.

Phase 3 — Berechnung und Export (Schritte 15–20): JaxFEM-Solver, Spannungsberechnung, Visualisierung, Ergebnisrückfluss in den Graph, LLM-Prompt-Generierung.

```
IFC-Datei (IfcOpenShell)
   +-- [Schritt 2] Bauteil-Parsing (IfcWall, IfcColumn, IfcSlab, ...)
   |       +-- BBox + Schwerpunkt (Weltkoordinaten, Z-normiert, m)
   |       +-- Material: E, nu aus Pset_MaterialMechanical
   |       +-- Synthetische Punktwolke (800 Punkte/Bauteil, DBSCAN-Eingabe)
   +-- [Schritt 5] IFC-Relationship-Analyse
   |       +-- IfcRelAggregates              -> part_of / contains
   |       +-- IfcRelContainedInSpatialStr.  -> hosted_in / Stockwerk
   |       +-- IfcRelAssociatesMaterial      -> shared_material
   +-- [Schritt 6] DBSCAN-Clustering (pro semantischem Label)
   |       +-- IFC-BBox-Snap (Schwerpunkt-Matching)
   +-- [Schritt 7] Feature-Extraktion: Volumen, Hoehe, Kompaktheit
   +-- [Schritt 8] Geometrische Beziehungsanalyse
   |       +-- above / below (dz > 0.5 m)
   |       +-- adjacent (BBox-Toleranz 5 cm)
   |       +-- inside / contains (BBox-Einschachtelung)
   |       +-- near (Distanz < REL_THRESHOLD)
   +-- [Schritt 9] NetworkX DiGraph (Scene Graph G)
   |       +-- [Schritt 11] Graphanalyse -> Einspannknoten + Lastflaechen
   |       +-- [Schritt 12] Material-Mapping: E_eff, nu_eff
       (volumengewichtet)
   +-- [Schritt 13] GMSH Tet4-Netz
   |       +-- IFC-Dreiecke via ifcopenshell.geom (STL-Binary)
   |       +-- ClassifySurfaces + CreateGeometry -> Volumen
   |       +-- fragment() -> Knoten-Merge an Kontaktflaechen
   +-- [Schritt 14] Randbedingungen aus Kontaktzonen
   +-- [Schritt 15] JaxFEM Solver (lineare 3D-Elastizitaet, UMFPACK)
   +-- [Schritt 16] Tet4-Spannungen (Knotenaveragierung)
   +-- [Schritt 18] Ergebnisse -> Scene Graph (pro Knoten)
   +-- [Schritt 19] LLM-Prompt (strukturierter Text fuer Spatial AI)
```

Abbildung 1: Vollständige Pipeline `IFC_SceneGraph_FEM.ipynb` mit Schrittverweisen

Einheitensystem: Alle Größen sind in **kN** und **m** gehalten (E-Modul in kN/m², Verschiebungen im Solver in m, Ausgabe in mm). Dies entspricht der im Tragwerkslabor üblichen Praxis.

3 KONFIGURATIONSPARAMETER (SCHRITT 1)

Alle Steuerparameter sind in einer einzigen Zelle gebündelt. Dies ermöglicht reproduzierbare Berechnungen durch einfaches Anpassen eines einzelnen Codeblocks.

Tabelle 2: Zentrale Konfigurationsparameter (Schritt 1)

Parameter	**Standardwert**	**Bedeutung und Empfehlung**
`USE_DEMO_MODEL`	`False`	`True`: synthetisches Demo-IFC (2 Wände, 1 Stütze, 1 Decke). `False`: Pfad `IFC_PATH` einlesen.
`IFC_PATH`	Duplex-Datei	Absoluter Pfad zur eigenen IFC-Datei.
`MESH_SIZE`	1,0 m	Maximale Tet4-Elementgröße. Für Produktionsberechnungen: 0,2–0,5 m.
`E_DEFAULT`	30 000 000 kN/m²	Fallback-E-Modul (C25/30 = 30 GPa), falls IFC keine Materialkennwerte enthält.
`NU_DEFAULT`	0,20	Fallback-Querdehnzahl.
`GAMMA_BETON`	25,0 kN/m³	Wichte für Eigengewichtsberechnung.
`Q_NUTZLAST`	5,0 kN/m²	Charakteristische Nutzlast nach DIN EN 1991-1-1.
`DBSCAN_EPS`	0,6 m	Nachbarschaftsradius für DBSCAN (Clustering-Empfindlichkeit). Zu klein: Cluster werden aufgespalten. Zu groß: Bauteile werden zusammengeführt.
`DBSCAN_MIN_SAMPLES`	5	Mindestpunkte pro Cluster (Rauschunterdrückung).
`REL_THRESHOLD`	5,0 m	Maximaler Schwerpunktabstand, bis zu dem eine geometrische Graphkante angelegt wird. Verhindert globale Vollvernetzung des Graphen.
`ADJ_TOLERANCE`	0,05 m	BBox-Berührungs-Toleranz für `adjacent`-Kanten (5 cm für Fugentoleranzen).

Das semantische Mapping von IFC-Klassen auf interne Labels ist als Dictionary `IFC_LABEL_MAP` definiert:

```
IFC_LABEL_MAP = {
    'IfcWall'             : 'wall',
    'IfcWallStandardCase' : 'wall',
    'IfcColumn'           : 'column',
    'IfcSlab'             : 'slab',
    'IfcBeam'             : 'beam',
    'IfcFooting'          : 'footing',
    'IfcPile'             : 'pile',
}
```

Listing 1: IFC-Klassen-Mapping

Nicht in dieser Liste enthaltene IFC-Typen (Fenster, Türen, Treppen, Installationen) werden vom Parsing ignoriert — dies ist eine bewusste Designentscheidung für die reine Tragwerksanalyse.

4 PHASE 1: MODELLAUFBAU

4.1 IFC laden und Demo-Modell (Schritt 2)

4.1.1 Demo-Modell

Wenn `USE_DEMO_MODEL = True`, erzeugt die Funktion `create_demo_ifc()` ein synthetisches IFC4-Modell mit `ifcopenshell.api`: zwei parallele Wände ($4{,}0 \times 0{,}2 \times 2{,}8\,\mathrm{m}$, bei $y = 0$ und $y = 3{,}8\,\mathrm{m}$), eine mittige Stütze ($0{,}2 \times 0{,}2 \times 2{,}8\,\mathrm{m}$) und eine Decke ($4{,}0 \times 4{,}0 \times 0{,}2\,\mathrm{m}$ auf $z = 2{,}8\,\mathrm{m}$). Alle Bauteile erhalten `Pset_MaterialMechanical`-Properties ($E = 30\,000\,\mathrm{kN/m^2}$ bzw. $35\,000\,\mathrm{kN/m^2}$ für die Stütze, $\nu = 0{,}2$). Das Demo-Modell wird in eine temporäre IFC-Datei geschrieben und dann über denselben Parsing-Pfad wie eine echte IFC-Datei eingelesen.

4.1.2 Eigene IFC-Datei und Z-Normalisierung

Bei `USE_DEMO_MODEL = False` wird `IFC_PATH` geöffnet. Vor dem Bauteil-Parsing erfolgt ein zweistufiges Vorgehen: In **Pass 1** werden alle BBoxen kurz eingelesen, um das globale Minimum der z-Koordinate aller Bauteile zu bestimmen. Dieser Wert `Z_GLOBAL_MIN` wird von allen z-Koordinaten subtrahiert, sodass das Gebäude mit dem Fußpunkt bei $z = 0$ beginnt. Dies ist entscheidend für die spätere automatische Erkennung der Einspannebene.

4.2 Bauteil-Parsing und Punktwolkenerzeugung (Schritt 3)

Für jedes tragwerksrelevante Element der IFC-Datei werden in **Pass 2** folgende Größen extrahiert:

1. **Semantisches Label** über `IFC_LABEL_MAP`
2. **Bounding Box** aus den Mesh-Vertices via `ifcopenshell.geom.create_shape()` mit `USE_WORLD_COORDS = True`: $\mathbf{b}_{\min}$, $\mathbf{b}_{\max}$ (Z-normiert, m)
3. **Materialkennwerte** E und ν aus `Pset_MaterialMechanical.YoungModulus` / `PoissonRatio` (Fallback: `E_DEFAULT`, `NU_DEFAULT`)
4. **Synthetische Punktwolke**: Gleichförmig verteilte Punkte in der BBox (Zielzahl 800, skaliert mit dem Volumenanteil, Minimum 50 Punkte)

Elemente mit degenerierter BBox (Volumen $< 10^{-6}\,\mathrm{m^3}$) oder fehlgeschlagener Geometrieerzeugung werden übersprungen. Die Punktwolken aller Bauteile werden zu einer gemeinsamen Punktwolke `cloud_pts` (Shape: $N \times 3$) und einem Label-Array `cloud_labels` zusammengeführt — die Eingabe für DBSCAN in Schritt 6.

4.3 IFC-Relationship-Analyse (Schritt 5)

Die Funktion `extract_ifc_relationships()` liest drei Typen von IFC-Beziehungen aus:

Tabelle 3: Extrahierte IFC-Beziehungstypen

IFC-Beziehung	Kanten-Typ	Semantik
`IfcRelAggregates`	`contains` / `part_of`	Hierarchische Zerlegung (z. B. Wand enthäl
`IfcRelContainedInSpatialStructure`	`hosted_in`	Zuordnung zu Stockwerk/Gebäude
`IfcRelAssociatesMaterial`	`shared_material`	Bauteile mit identischem Material

Zur Auflösung der Beziehungen auf interne Schlüssel wird eine GUID-Lookup-Tabelle verwendet. In der Duplex-Beispieldatei liefert `IfcRelAggregates` keine direkten Bauteil-zu-Bauteil-Kanten (der Wert ist 0 IFC-Kanten). Dies ist modellspezifisch und kein Algorithmusfehler: Das Duplex-Modell verwendet keine explizite Aggregationshierarchie zwischen den tragwerksrelevanten Elementen. Alle 24 Kanten des finalen Graphen stammen daher aus der geometrischen Beziehungsanalyse (Schritt 8).

4.4 Semantisches DBSCAN-Clustering (Schritt 6)

4.4.1 Clustering-Algorithmus

Analog Poux & Lehtola [1] wird DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) pro semantischem Label auf die zugehörigen Punkte der gemeinsamen Punktwolke angewendet. DBSCAN wurde gegenüber k-Means gewählt, da es keine a priori bekannte Clusteranzahl benötigt und beliebig geformte Cluster sowie Rauschen (Label $= -1$, d. h. Ausreißer) behandeln kann. Die Parameter `DBSCAN_EPS` (Nachbarschaftsradius) und `DBSCAN_MIN_SAMPLES` steuern die Sensitivität.

4.4.2 IFC-BBox-Snap

Da die Punktwolken synthetisch aus den IFC-BBoxen erzeugt wurden, ist der Clustering-Schritt für echte IFC-Daten redundant — die Instanzsegmentierung ist bereits durch das IFC-Parsing gegeben. In diesem Fall (`USE_DEMO_MODEL = False`) wird das Clustering daher übersprungen und jedes Element direkt als eigener Cluster angelegt. Für Demo-Daten gleicht der anschließende `snap_to_ifc_bboxes()`-Schritt Cluster-Schwerpunkte auf die nächstgelegenen exakten IFC-BBoxen ab und übernimmt die Materialkennwerte.

4.5 Feature-Extraktion (Schritt 7)

Für jeden Cluster-Knoten werden geometrische Merkmale berechnet:

Tabelle 4: Extrahierte Objektfeatures

Feature	Formel	Bedeutung
V	$\Delta x \cdot \Delta y \cdot \Delta z$	BBox-Volumen (m^3)
SA	$\text{ConvexHull}(\{p_i\}).area$	Konvexe Hüllfläche (m^2)
C	$SA^3/(36\pi V^2)$	Kompaktheit (1 = Kugel)
h	Δz	Höhe (m)
AR	$\max(\Delta)/\min(\Delta)$	Aspektverhältnis

Die Kompaktheit C ist ein dimensionsloses Maß für die Schlankeit eines Bauteils: Eine Kugel hat $C = 1$, flache Decken haben sehr kleine C-Werte, schlanke Stützen hohe AR-Werte. Diese Features werden den Graphknoten als Attribute übergeben und können für LLM-Abfragen oder spätere Klassifikatoren genutzt werden.

4.6 Räumliche Beziehungsanalyse (Schritt 8)

Der Drei-Stufen-Algorithmus in `classify_spatial_rel()` prüft für jedes Bauteilpaar (i, j) mit Schwerpunktabstand $d_{ij} < d_{\text{rel}}$:

1. **Vertikal:** Falls $|\Delta z| > 0{,}5\,\text{m}$ → `above` (obj_i über obj_j) oder `below`
2. **Containment:** Liegt BBox_i vollständig in BBox_j? → `inside` / `contains`
3. **Adjacency:** Berühren sich die BBoxen mit Toleranz $\varepsilon_{\text{adj}} = 5\,\text{cm}$? → `adjacent`
4. **Fallback:** Keine der obigen → `near`

Die geometrischen Relationships werden mit den IFC-Relationships aus Schritt 5 kombiniert, wobei bereits vorhandene Paare nicht doppelt eingetragen werden. Das Ergebnis ist eine Liste aller Kanten mit Typ und Quelle.

5 SCENE GRAPH — FORMALE BESCHREIBUNG (SCHRITT 9)

5.1 Graphstruktur

Der Scene Graph $G = (V, E)$ ist ein gerichteter Graph, implementiert als NetworkX `DiGraph`.

Knotenmenge V: Jeder Knoten $v_i \in V$ repräsentiert eine Bauteilinstanz mit folgenden Attributen:

$$v_i = \big(\text{label},\ \text{ifc_name},\ \text{ifc_guid},\ \mathbf{x}_c,\ \mathbf{b}_{\min},\ \mathbf{b}_{\max},\ V_i,\ h_i,\ C_i,\ E_i,\ \nu_i,\ \ldots\big)$$

Kantenmenge E: Jede Kante $e_{ij} \in E$ trägt Beziehungstyp `relationship` und Quelle `source` $\in$ {`geometric`, `ifc`}:

Tabelle 5: Kanten-Typen im Scene Graph

Typ	Quelle	Bedeutung und Ableitung
`above` / `below`	geometrisch	Vertikale Lagebeziehung ($\lvert\Delta z\rvert > 0{,}5\,\mathrm{m}$ zwischen Schwerpunkten)
`adjacent`	geometrisch	BBox-Abstand $< \varepsilon_{\mathrm{adj}} = 5\,\mathrm{cm}$ in mind. einer Achse
`near`	geometrisch	Schwerpunktabstand $< d_{\mathrm{rel}}$, keine engere Relation
`inside` / `contains`	geometrisch	Vollständige BBox-Einschachtelung
`part_of` / `contains`	IFC	`IfcRelAggregates`: hierarchische Zerlegung
`hosted_in`	IFC	`IfcRelContainedInSpatialStructure`: Stockwerkzuordnung
`shared_material`	IFC	`IfcRelAssociatesMaterial`: gleiche Materialspezifikation

5.2 Graph-Metriken und Analysefunktionen (Schritt 10–11)

Die Funktion `analyze_graph()` berechnet folgende Metriken:

- Anzahl schwach zusammenhängender Komponenten (`nx.number_weakly_connected_components`)
- Verteilung der semantischen Labels auf Knoten
- Verteilung der Kantentypen
- IFC-Kanten- vs. Geometrie-Kanten-Anteil
- Automatische Identifikation von **Einspannknoten** und **Lastflächenknoten**

5.3 Automatische Ableitung der FEM-Randbedingungen

Die topologiebasierte BC-Ableitung ist das Herzstück der Automatisierungslogik:

Tabelle 6: Graphtopologie → FEM-Randbedingungen

Graph-Eigenschaft	FEM-Ableitung
`wall`, `column` oder `footing` ohne eingehende `below`-Kante	Einspannung am Fußpunkt: $\mathbf{u} = \mathbf{0}$ auf der Ebene $z = z_{\min}$ dieses Elements
`slab` mit ausgehender `above`- oder `adjacent`-Kante	Lastauflagefläche: Neumann-BC $\mathbf{q}$ auf Oberfläche
Kein `slab` als Lastfläche identifiziert	Fallback: alle Knoten mit Label `slab` werden als Lastfläche verwendet

Diese Regeln basieren auf dem Prinzip der *geschlossenen Lastpfade*: Ein tragendes Element ohne Auflager darunter muss am Boden eingespannt sein; eine Decke, die über anderen Elementen liegt, ist die primäre

Lastauflagefläche. Die Ableitung ist modellspezifisch robust, solange das IFC-Modell eine konsistente vertikale Geometrie aufweist.

6 PHASE 2: FEM-VORBEREITUNG

6.1 Material-Mapping (Schritt 12)

Aus den Graphknoten-Attributen wird der volumengewichtete effektive E-Modul und die effektive Querdehnzahl nach der Voigt-Schranke (obere Schranke der Homogenisierungstheorie, Iso-Dehnungsannahme) berechnet:

$$E_{\text{eff}} = \frac{\sum_i E_i \cdot V_i}{\sum_i V_i}, \qquad \nu_{\text{eff}} = \frac{\sum_i \nu_i \cdot V_i}{\sum_i V_i} \tag{1}$$

Daraus folgen die Lamé-Parameter für den Solver:

$$\mu = \frac{E_{\text{eff}}}{2(1+\nu_{\text{eff}})}, \qquad \lambda = \frac{E_{\text{eff}}\,\nu_{\text{eff}}}{(1+\nu_{\text{eff}})(1-2\nu_{\text{eff}})} \tag{2}$$

Die Voigt-Formulierung ist eine konservative Näherung: Sie überschätzt die Steifigkeit heterogener Verbundsysteme (Stahl-Beton-Verbund, Holz-Beton-Verbunddecken). Für homogene Stahlbetongebäude wie das Duplex-Modell — alle Bauteile haben dasselbe $E_i = 30\,000\,000\,\text{kN/m}^2$ — liefert sie das exakte Ergebnis ($E_{\text{eff}} = 30\,000\,000\,\text{kN/m}^2$). Die Erweiterung auf bauteilweises heterogenes Materialfeld ist Prio 2 der Roadmap.

6.2 GMSH Tet4-Netz aus IFC-Geometrie (Schritt 13)

6.2.1 Strategie: STL-Merge

Die Funktion `build_tet4_from_ifc()` erzeugt das Volumennetz in sechs Teilschritten:

1. **Geometrie-Export:** Für jedes IFC-Element werden die Dreiecke der Oberfläche via `ifcopenshell.geom.create_sh` mit `USE_WORLD_COORDS = True` eingelesen (Vertices + Faces-Array).
2. **Z-Normalisierung:** Alle z-Koordinaten werden um `Z_GLOBAL_MIN` verschoben.
3. **STL-Binary-Schreiben:** Die Dreiecke werden ohne externes Paket (nur `struct.pack`) als binäre STL-Datei in eine temporäre Datei geschrieben.
4. **GMSH-Merge:** `gmsh.merge(stl)` lädt die Oberflächentriangulierung. `ClassifySurfaces()` klassifiziert planare Patches; `CreateGeometry()` schließt die Oberfläche zu einem Volumen.
5. **Fragment:** `gmsh.model.occ.fragment()` verbindet alle Volumen an gemeinsamen Kontaktflächen. Dies ist der kritische Schritt für die spätere Knotenkompatibilität an Wand-Decken-Übergängen: Ohne Fragment werden an den Schnittkanten doppelte Knoten erzeugt, was zu Sprüngen im Verschiebungsfeld führt.
6. **Vernetzung:** `gmsh.model.mesh.generate(3)` erzeugt das Tet4-Netz mit Elementgröße `MESH_SIZE`. Algorithm3D = 4 (Frontal-Delaunay) liefert qualitativ hochwertige Tetraeder.

6.2.2 Fallback-Strategie

Wenn `ClassifySurfaces` für ein Element kein geschlossenes Volumen erzeugt (offene STL-Geometrie, degenerierende Dreiecke), wird automatisch auf eine BBox-Approximation zurückgefallen. Dies wird mit einem Hinweis protokolliert. In der Praxis tritt der Fallback bei IFC-Dateien mit fehlendem oder unvollständigem Geometrie-Export auf.

6.2.3 Netzausgabe

Das fertige Netz wird mit `meshio` als `.msh`-Datei (GMSH-Format) und als `.vtu`-Datei (VTK-Format, für Paraview/PyVista) exportiert. Für JaxFEM werden extrahiert:

- `points_m` — Knotenkoordinaten (Shape: $N_K \times 3$, in m)
- `cells_m` — Konnektivitätstabelle (Shape: $N_E \times 4$, Integer-Indizes)

6.3 Randbedingungen (Schritt 14)

6.3.1 Kontaktzonenbasierte BCs

Für jede `adjacent`-, `above`- oder `below`-Kante im Scene Graph wird die Schnitt-BBox der beiden beteiligten Bauteile berechnet:

$$\mathbf{b}^{\cap}_{\min} = \max(\mathbf{b}^{A}_{\min}, \mathbf{b}^{B}_{\min}) - \varepsilon, \qquad \mathbf{b}^{\cap}_{\max} = \min(\mathbf{b}^{A}_{\max}, \mathbf{b}^{B}_{\max}) + \varepsilon$$

mit $\varepsilon = \texttt{MESH_SIZE}/4$. Diese Schnitt-BBox definiert die Kontaktzone. Der BC-Typ wird aus den Materialkennwerten abgeleitet:

- **Holzkonstruktionen** (Stichwortsuche in Materialname): `pinned` (gelenkige Verbindung)
- **Stahlbeton + Stahlbeton**: `fixed` (eingespannt, monolithischer Verbund)
- **Fallback** ($\mathrm{E} < 15\,000\,000\,\mathrm{kN/m^2}$): `pinned`

6.3.2 JAX-kompatible Location-Funktionen

Für JaxFEM werden die Randbedingungen als Python-Funktionen `location_fns` übergeben, die JAX-seitig auf die Knotenkoordinaten angewendet werden. Alle Berechnungen innerhalb dieser Funktionen müssen `jnp`-Operationen sein (kein NumPy, kein KD-Tree), da sie im JAX-Tracing-Kontext ausgeführt werden. Die Dirichlet-BCs werden als `dirichlet_bc_info`-Dictionary mit den zugehörigen Wert-Funktionen übergeben.

7 FEM-FORMULIERUNG (SCHRITT 15–16)

7.1 Konstitutives Gesetz — Lineare 3D-Elastizität

Das Materialverhalten wird durch das isotrope Hooke'sche Gesetz beschrieben. Der Cauchy-Spannungstensor ist:

$$\boldsymbol{\sigma} = \lambda \,\mathrm{tr}(\boldsymbol{\varepsilon})\,\mathbf{I} + 2\mu\,\boldsymbol{\varepsilon} \tag{3}$$

mit dem symmetrischen Kleinschen Verzerrungstensor:

$$\boldsymbol{\varepsilon}(\mathbf{u}) = \frac{1}{2}\big(\nabla\mathbf{u} + (\nabla\mathbf{u})^{\top}\big) \tag{4}$$

7.2 Schwache Form und FEM-Diskretisierung

Das zu lösende Randwertproblem lautet: Gesucht ist das Verschiebungsfeld $\mathbf{u} : \Omega \to \mathbb{R}^3$ mit

$$-\mathrm{div}\,\boldsymbol{\sigma}(\mathbf{u}) = \mathbf{0} \quad \text{in } \Omega \tag{5}$$

$$\mathbf{u} = \mathbf{0} \quad \text{auf } \Gamma_D \text{ (Einspannflächen)} \tag{6}$$

$$\boldsymbol{\sigma}(\mathbf{u})\,\mathbf{n} = -q_{\mathrm{ges}}\,\mathbf{e}_z \quad \text{auf } \Gamma_N \text{ (Lastfläche)} \tag{7}$$

Die Gesamtlast setzt sich zusammen aus Eigengewicht und Nutzlast:

$$q_{\mathrm{ges}} = \gamma_{\mathrm{Beton}} \cdot h_{\mathrm{Decke}} + q_{\mathrm{Nutz}} = 25{,}0 \cdot 0{,}2 + 5{,}0 = 10{,}0 + 5{,}0 = 15{,}0 \cdot (\text{Deckenhöhe variabel})\ \mathrm{kN/m^2} \tag{8}$$

Im Duplex-Beispiel ergibt sich $q_{\mathrm{ges}} = 21{,}81\,\mathrm{kN/m^2}$ (abhängig von der gemittelten Deckenhöhe des Modells).

7.3 Tet4-Element

Das lineare Tetraederelement (Tet4) hat 4 Knoten und $4 \times 3 = 12$ Freiheitsgrade. Die bilinearen Formfunktionen in baryzentrischen Koordinaten $(\xi_1, \xi_2, \xi_3, \xi_4)$ mit $\sum \xi_i = 1$ lauten:

$$N_1 = \xi_1, \quad N_2 = \xi_2, \quad N_3 = \xi_3, \quad N_4 = \xi_4$$

Die Transformation auf physikalische Koordinaten erfolgt über die Jacobi-Matrix:

$$\mathbf{J} = \begin{bmatrix} \mathbf{p}_2 - \mathbf{p}_1 \\ \mathbf{p}_3 - \mathbf{p}_1 \\ \mathbf{p}_4 - \mathbf{p}_1 \end{bmatrix}^\top \in \mathbb{R}^{3\times 3} \tag{9}$$

Die Formfunktions-Gradienten im physikalischen Raum sind:

$$\frac{\partial \mathbf{N}}{\partial \mathbf{x}} = \frac{\partial \mathbf{N}}{\partial \boldsymbol{\xi}} \cdot \mathbf{J}^{-1} \tag{10}$$

Das Tet4-Element hat einen konstanten Verzerrungszustand im gesamten Element (constant strain tetrahedron). Für eine genaue Erfassung von Spannungsgradienten sind daher feine Netze erforderlich — dies ist der Hauptgrund für die Empfehlung `MESH_SIZE` $\leq 0{,}3\,\mathrm{m}$ in zukünftigen Berechnungen.

7.4 JaxFEM-Problemklasse

Die Materialformulierung wird in JaxFEM als Python-Klasse definiert, die von `Problem` erbt:

```
class IFC_SG_FEM(Problem):
    def get_tensor_map(self):
        def stress(u_grad):
            eps = 0.5 * (u_grad + u_grad.T)
            return lmbda * jnp.trace(eps) * jnp.eye(self.dim) \
                   + 2.0 * mu * eps
        return stress

    def get_surface_maps(self):
        def load_top(u, x):
            return jnp.array([0.0, 0.0, q])    # Drucklast in -z
        return [load_top]
```

Listing 2: JaxFEM-Problemklasse IFC_SG_FEM

`get_tensor_map()` definiert die Voigt-artige Materialmatrix; `get_surface_maps()` liefert die Neumann-Randbedingung als Vektorfunktion. Der Solver wird mit UMFPACK als direktem Gleichungssystemlöser aufgerufen: `solver(problem, solver_options={'umfpack_solver': {}})`.

7.5 Von-Mises-Vergleichsspannung und Spannungsberechnung (Schritt 16)

Die Von-Mises-Vergleichsspannung ist definiert als:

$$\sigma_{\mathrm{vM}} = \sqrt{\frac{3}{2}\,\mathbf{s} : \mathbf{s}}, \qquad \mathbf{s} = \boldsymbol{\sigma} - \frac{1}{3}\,\mathrm{tr}(\boldsymbol{\sigma})\,\mathbf{I} \tag{11}$$

Die Funktion `compute_stress_tet4()` implementiert eine Knotenaveragierung: Für jedes Element wird der Cauchy-Spannungstensor aus dem Verschiebungsgradient berechnet (ein Gauss-Punkt im Mittelpunkt des Tet4), und die 6 unabhängigen Komponenten $(\sigma_{11}, \sigma_{22}, \sigma_{33}, \sigma_{12}, \sigma_{13}, \sigma_{23})$ werden zu den anliegenden Knoten akkumuliert und am Ende durch die Anzahl der anliegenden Elemente dividiert. Dies entspricht einer einfachen Knotenaveragierung (keine SPR-Glättung).

8 ERGEBNISSE — ERSTER MEILENSTEIN (SCHRITT 17–18)

8.1 Modell-Kenngrößen

Getestet mit der öffentlich verfügbaren Datei `Ifc2x3_Duplex_Architecture.ifc`, einem zweigeschossigen Wohngebäude aus dem buildingSMART-Datensatz.

Tabelle 7: Modell- und Netz-Kenngrößen

Kenngröße	**Wert**
IFC-Bauteile (gesamt geparst)	149
Graph-Knoten (nach Clustering/Bypass)	13
Graph-Kanten (gesamt)	24
davon IFC-Kanten	0
davon geometrische Kanten	24
Einspannknoten (foundation_nodes)	variabel (walls + columns ohne below)
Lastflächenknoten (load_surface_nodes)	alle slabs
Tet4-Elemente	1 958
FEM-Knoten	639
Freiheitsgrade (DOF)	1 917
Effektiver E-Modul E_{eff}	30 000 000 kN/m²
Effektive Querdehnzahl ν_{eff}	0,20
Gesamtlast q_{ges}	21,81 kN/m²

8.2 FEM-Ergebnisse

Tabelle 8: FEM-Ergebnisse (Meilenstein 1, `MESH_SIZE` = 1,0 m)

Größe	**Symbol**	**Wert**	**Einheit**
Maximale z-Absenkung	$u_{z,\min}$	−2,064	mm
Maximale Gesamtverschiebung	$\lvert\mathbf{u}\rvert_{\max}$	2,065	mm
Maximale Von-Mises-Spannung	$\sigma_{\text{vM,max}}$	7 529,7	kN/m²
Minimale Druckspannung	$\sigma_{zz,\min}$	−9 072,6	kN/m²

8.3 Plausibilitätsbewertung

Verschiebungen: Die maximale Durchbiegung von $u_{z,\min} = -2{,}064$ mm ist mit der groben Netzauflösung ($h = 1$ m) konsistent. Für eine C25/30-Stahlbetondecke mit $q = 21{,}81$ kN/m² und einer Spannweite im Bereich von $l \approx 4$ m wäre der theoretische Grenzwert nach DIN EN 1990 ($l/300 \approx 13$ mm) nicht überschritten — was für ein Grobnetz mit eher konservativer (steifer) Abschätzung plausibel ist.

Spannungen: $\sigma_{\text{vM,max}} = 7\,530$ kN/m² = 7,53 MPa liegt deutlich unter der charakteristischen Betondruckfestigkeit $f_{ck} = 25$ MPa (C25/30). Der Ausnutzungsgrad beträgt ca. 30%, was für eine Grobnetz-Rechnung mit konservativem Materialgesetz plausibel ist. Die minimale Druckspannung $\sigma_{zz,\min} = -9{,}1$ MPa entspricht ca. 36% von f_{ck} — ebenfalls im Bereich einer nicht ausgenutzten Konstruktion.

Vorzeichenkonsistenz: Der in der Implementierung vorhandene Assertion-Test `assert uz.min() < 0` stellt sicher, dass eine Absenkung (negatives u_z) vorhanden ist. Würde das Vorzeichen der Lastfunktion in `get_surface_maps()` vertauscht sein, würde dieser Test fehlschlagen.

Konvergenzhinweis: Für eine gesicherte Aussage ist ein Konvergenztest mit abgestuften Netzfeinheiten ($h = 1{,}0 \to 0{,}5 \to 0{,}3 \to 0{,}2$ m) erforderlich (Prio 1 der Roadmap). Das Tet4-Element mit konstantem

Dehnungsfeld neigt zur Überschätzung der Steifigkeit bei groben Netzen.

8.4 Ergebnisrückfluss in den Scene Graph (Schritt 18)

Die Funktion `write_fem_to_graph()` weist jedem Graphknoten die FEM-Ergebnisse des zugehörigen räumlichen Bereichs zu: Für jeden Knoten werden alle FEM-Netzknoten innerhalb der Bauteil-BBox (mit Toleranz 10^{-6} m) identifiziert und darüber $u_{z,\min}$, $|\mathbf{u}|_{\max}$ und $\sigma_{\mathrm{vM,max}}$ berechnet. Damit wird der ursprünglich geometrisch-semantische Scene Graph zu einem vollständigen **strukturmechanischen Wissensgraphen** erweitert, der für die weitere Auswertung — insbesondere die LLM-Integration — zur Verfügung steht.

9 LLM-INTEGRATION (SCHRITT 19)

9.1 Prompt-Generierung

Die Funktion `graph_to_llm_prompt()` serialisiert den angereicherten Scene Graph in einen strukturierten Textprompt. Dieser enthält für jeden Knoten alle Attribute (IFC-Name, Klasse, Typ, Schwerpunkt, Volumen, E-Modul, $u_{z,\min}$, $\sigma_{\mathrm{vM,max}}$) sowie alle Kanten mit Beziehungstyp und Quelle. Abschließend werden die globalen FEM-Kenngrößen angefügt.

```
=== IFC TRAGWERK - SCENE GRAPH + FEM-ERGEBNISSE ===

BAUTEILE (Knoten):
  [wall_0]  IFC=BasicWall:... (IfcWall)  Typ=wall
            Schwerpunkt=(1.2, 0.1, 1.4) m  Volumen=3.136 m3
            E=30000000 kN/m2  uz_min=-1.2 mm  sigma_vM_max=4200.0 kN/m2

BEZIEHUNGEN (Kanten):
  wall_0  --[adjacent | geometric]-->  slab_0
  slab_0  --[above    | geometric]-->  wall_1

FEM-ERGEBNISSE (Global):
  Max. Absenkung   : -2.064 mm
  Max. Verformung  :  2.065 mm
  Max. sigma_vM    : 7529.7 kN/m2
  Gesamtlast       : 21.81 kN/m2
```

Listing 3: Schema des LLM-Prompts (graph_to_llm_prompt)

9.2 Anwendungsbeispiele

Dieser Prompt ermöglicht einem LLM (z. B. Claude) die Beantwortung strukturmechanischer Fragen über das Gebäudemodell:

- „Welches Bauteil hat die größte Von-Mises-Spannung, und was ist strukturmechanisch der Grund dafür?"
- „Ist die Durchbiegung der Decke normkonform nach DIN EN 1990 Tabelle B.2 ($u_z \leq l/300$)?"
- „Welche Bauteile sind direkt über der Stütze und nehmen deren Drucklast auf?"
- „In welchem Stockwerk liegt der kritisch beanspruchte Bereich?"
- „Welche Maßnahmen könnten die kritische Spannung um 20% reduzieren?"

9.3 API-Anbindung

Der Prompt-Export ist implementiert; der automatische API-Call ist als Prio 4 in der Roadmap vorgesehen (Abschnitt 12):

```
import anthropic
client = anthropic.Anthropic()
response = client.messages.create(
```

```
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[{
        "role": "user",
        "content": prompt + "\n\nFRAGE: " + question
    }]
)
```

Listing 4: Geplante Claude-API-Anbindung (Prio 4)

10 VERGLEICH: VORGÄNGER VS. NEUES NOTEBOOK

Tabelle 9: IFC_FEM_3D.ipynb (Generation 1) vs. IFC_SceneGraph_FEM.ipynb (Generation 2)

Merkmal	**IFC_FEM_3D**	**IFC_SceneGraph_FEM**
Netzgeometrie	Achsenparallele BBox-Approximation	**Echte IFC-Triangulierung** (STL-Merge via ifcopenshell.geom)
Randbedingungen	Manuell festgelegte z-Ebene	**Automatisch aus Graph-Topologie**
Lastflächen	Manuell definiert	**Automatisch aus Topologie**
Material	Manuell (ein globaler E-Wert)	**Automatisch aus IFC-Psets, volumengewichtet**
Struktursemantik	Keine	**NetworkX DiGraph, 7 Kantentypen**
IFC-Relationships	Keine	**IfcRelAggregates, IfcRelAssociatesMaterial**
DBSCAN-Clustering	Keine	**Pro semantischem Label**
Feature-Extraktion	Keine	**Volumen, Höhe, Kompaktheit, Aspektverhältnis**
Kontaktzonen-BCs	Keine	**Material-basiert (Holz: gelenkig, Beton: eingespannt)**
LLM-Export	Keine	**Strukturierter Textprompt**
Ergebnisrückfluss	Keine	**FEM-Werte in Graphknoten**
JSON-Export	Keine	**Vollständiger Scene-Graph-JSON**

11 BEKANNTE EINSCHRÄNKUNGEN (VERSION 1.0)

1. **Fehlende IFC-Kanten im Duplex-Beispiel:** Das Modell `Ifc2x3_Duplex_Architecture.ifc` enthält keine `IfcRelAggregates`-Beziehungen zwischen tragwerksrelevanten Bauteilen (0 IFC-Kanten im finalen Graphen). Alle 24 Kanten sind geometrischer Herkunft. Der Algorithmus ist korrekt implementiert; das Problem liegt im Quellmodell. Bei IFC4-Dateien mit vollständig befüllten Relationships (z. B. aus Revit-Exports mit Structural Analysis Extension) ist ein deutlich reichhaltigerer Graph zu erwarten.

2. **Grobe Netzauflösung:** `MESH_SIZE = 1,0 m` liefert nur 1 958 Elemente und ist für quantitative Aussagen zu grob. Die Tet4-Spannungen konvergieren langsam; bei $h = 1\,\mathrm{m}$ ist die Spannung um den

Faktor 2–5 vom Konvergenzwert entfernt. Empfehlung für Produktionsberechnungen: $h = 0{,}2$–$0{,}3\,\mathrm{m}$ (entspricht ca. 50 000–200 000 Elemente für das Duplex-Modell).

3. **Voigt-Homogenisierung:** Das volumengewichtete E_{eff} ist eine obere Schranke. Für reine Stahlbeton-Gebäude ist sie exakt; für Verbundkonstruktionen (Holz-Beton, Stahl-Beton) überschätzt sie die Steifigkeit. Bauteilweises heterogenes Materialfeld ist anzustreben.
4. **Nur LLE (lineare 3D-Elastizität):** Das Materialmodell berücksichtigt weder Plastizität, Rissbildung noch geometrische Nichtlinearität. Für die Bewertung des Grenzzustands der Tragfähigkeit nach DIN EN 1992 ist eine nichtlineare Berechnung erforderlich.
5. **LLM-Auto-Call noch nicht integriert:** Der Prompt wird generiert und ausgegeben; der automatische API-Call an Claude ist Prio 4 der Roadmap.
6. **Nur IFC2x3-Beispieldatei getestet:** Das Parsing wurde auf IFC2x3 und das Demo-Modell (IFC4) getestet. Die Robustheit gegenüber verschiedenen IFC-Exportern (Revit, ArchiCAD, FreeCAD) und IFC-Versionen ist noch zu validieren.

12 ROADMAP UND NÄCHSTE ENTWICKLUNGSSCHRITTE

Tabelle 10: Priorisierte Entwicklungsschritte

Prio	Schritt	Technisches Ziel und Motivation
1	Konvergenztest ($h = 1{,}0 \rightarrow 0{,}2\,\mathrm{m}$)	Quantitative Netzkonvergenzkurve für Verschiebungen und Spannungen; Bestimmung der notwendigen Netzfeinheit für normkonforme Aussagen.
2	Bauteilweises Materialfeld	Subregionale JaxFEM-Probleme oder feldweise E-Funktion; Reuss- und Voigt-Schranken vergleichen.
3	IFC4 mit befüllten Psets	Testen mit einem Revit-Export (IFC4 + `Pset_MaterialMechanical`); Validierung der automatischen Material-Extraktion.
4	LLM-Auto-Call via Anthropic API	Automatische Übergabe des LLM-Prompts an `claude-sonnet-4-6` nach erfolgreicher FEM-Berechnung; strukturierte Antwortparsing.
5	JSON → CesiumJS-Einfärbung	Export der σ_{vM}-Werte als JSON für CesiumJS-Einfärbung in geobim.app; Verknüpfung der FEM-Ergebnisse mit der geographischen Lage des Gebäudes.
6	Scan-to-BIM-Eingang	Direkteinspeisung einer Punktwolke (.las/.ply) über den PointCloudEditor (PicoGK-Pipeline) als IFC-Eingabe [9, 10].
7	Gradientenbasierte Formoptimierung	Nutzung der JAX-Differenzierbarkeit für Topologieoptimierung nach SIMP-Methode; direkt aus dem Notebook ohne Solver-Umbau.

13 SOFTWAREABHÄNGIGKEITEN UND UMGEBUNG

13.1 Python-Pakete

```
ifcopenshell >= 0.8   # IFC-Parsing, Geometrieerzeugung, Relationship-Analyse
```

```
jax, jaxlib            # Autodiff-Framework, JIT-Kompilierung
jax_fem                # FEM-Solver (git: tianjuxue/jax-fem)
numpy                  # Numerische Berechnungen (als onp importiert)
networkx               # Scene Graph (DiGraph), Graph-Analyse
scikit-learn           # DBSCAN-Clustering
scipy                  # ConvexHull (Feature-Extraktion), cKDTree
gmsh                   # Netzgenerierung (Tet4, Python-API)
meshio                 # Netzformat-Konvertierung (.msh -> .vtu)
pyvista                # 3D-Visualisierung, Warp-by-Vector
matplotlib             # 2D-Visualisierungen (Schnittbilder, Balkendiagramme)
tqdm                   # Fortschrittsanzeige
```

13.2 Conda-Umgebung

```
conda activate jaxfem   # Umgebung: environment.yml im JaxFEM-Root
```

Die Umgebung enthält alle Abhängigkeiten inkl. CUDA-kompatiblem JAX (falls GPU verfügbar). Für reine CPU-Ausführung genügt `jaxlib==0.4.x` (kein CUDA).

13.3 Ausgabedateien

Tabelle 11: Erzeugte Ausgabedateien

Datei	Inhalt
`data/ifc_scene_graph_fem.json`	Vollständiger Scene Graph mit FEM-Ergebnissen (JSON)
`data/vtk/ifc_sg_fem.vtu`	VTK-Netz mit allen Feldgrößen (Paraview-kompatibel)
`data/mesh.msh`	GMSH-Netz (für Nachbearbeitung mit GMSH-GUI)

14 ZUSAMMENFASSUNG ALLER NOTEBOOK-SCHRITTE

Tabelle 12: Vollständige Schritt-Übersicht (IFC_SceneGraph_FEM.ipynb)

Schritt	Bezeichnung	Methode / Ergebnis
0	Importe & Fallbacks	Python-Pakete; NetworkX-Fallback-Klasse falls nicht installiert
1	Konfiguration	Zentrales Parameter-Dictionary; `IFC_LABEL_MAP`
2	IFC laden	Demo-IFC oder eigene Datei; Z-Normalisierung auf $z = 0$
3	Bauteil-Parsing	BBox, Material, synthetische Punktwolke; $Z_{\text{global,min}}$-Normierung
4	Punktwolke visualisieren	XZ + YZ Schnittbilder (Matplotlib)
5	IFC-Relationship-Analyse	IfcRelAggregates, IfcRelAssociatesMaterial, Stockwerk-Map
6	DBSCAN-Clustering	Pro Label; IFC-BBox-Snap; Bypass für echte IFC-Daten
7	Feature-Extraktion	Volumen, Oberfläche, Kompaktheit, Höhe, Aspektverhältnis
8	Räumliche Beziehungen	above/below/adjacent/inside/near + IFC-Kanten
9	Scene Graph	NetworkX DiGraph; Knoten- und Kanten-Attribute
10	Graph visualisieren	XY-Grundriss + Federlayout (Matplotlib)
11	Graph-Analyse	Einspannknoten, Lastflächenknoten, Metrik-Report
12	Material-Mapping	E_{eff}, ν_{eff}, μ, λ (Voigt)
13	GMSH Tet4-Netz	STL-Merge, ClassifySurfaces, fragment(), Vernetzung
14	Randbedingungen	Kontaktzonen-BCs, material-basiert; JAX-location_fns
15	JaxFEM-Solver	Lineare 3D-Elastizität, UMFPACK; **u**-Lösung
16	Spannungen	Tet4-Knotenaveragierung; σ_{ij}, σ_{vM}
17	PyVista-Visualisierung	u_z, $\lvert\mathbf{u}\rvert$, σ_{vM}, σ_{zz}; Warp-Faktor-Plot
18	Ergebnisrückfluss	FEM-Werte in Graphknoten; JSON-Export
19	LLM-Prompt	Strukturierter Text; Beispielabfragen für Spatial AI
20	Zusammenfassung	Konsolen-Report aller Kenngrößen

14 REFERENZEN

14 LITERATUR

[1] Poux, F. & Lehtola, V. (2025). *3D Scene Graphs for Spatial AI and LLMs*. Data Science Collective.

[2] Chacón, R., Ramonell, C., Ventosa San Martino, M. & Posada, H. (2026). Semantic digital twins for masonry bridges: Structuring geometrical, material and assessment field data. *Engineering Structures*, 357, 122503. https://doi.org/10.1016/j.engstruct.2026.122503

[3] Zhai, R., Zou, J., He, Y. & Meng, L. (2022). BIM-driven data augmentation method for semantic segmentation in superpoint-based deep learning network. *Automation in Construction*, 140, 104373. https://doi.org/10.1016/j.autcon.2022.104373

[4] Jin, R., Zhong, B., Ma, L., Hashemi, A. & Ding, L. (2019). Integrating BIM with building performance analysis in project life-cycle. *Automation in Construction*, 106, 102861. https://doi.org/10.1016/j.autcon.2019.102861

[5] Alves, J. L., Palha, R. P. & Almeida Filho, A. T. (2025). Towards an integrative framework for BIM and artificial intelligence capabilities in smart architecture, engineering, construction, and operations projects. *Automation in Construction*, 174, 106168. https://doi.org/10.1016/j.autcon.2025.106168

[6] Peng, Y., Au-Yong, C. P. & Myeda, N. E. (2024). Knowledge graph of building information modelling (BIM) for facilities management (FM). *Automation in Construction*, 165, 105492. https://doi.org/10.1016/j.autcon.2024.105492

[7] Wang, M., Lilis, G. N., Mavrokapnidis, D., Katsigarakis, K., Korolija, I. & Rovas, D. (2024). A knowledge graph-based framework to automate the generation of building energy models using geometric relation checking and HVAC topology establishment. *Energy and Buildings*, 325, 115035. https://doi.org/10.1016/j.enbuild.2024.115035

[8] Wang, M., Lilis, G. N., Mavrokapnidis, D., Katsigarakis, K., Korolija, I. & Rovas, D. (2026). Automating HVAC enrichment in building energy models from imperfect BIM data through topology completeness validation. *Applied Energy*, 404, 127171. https://doi.org/10.1016/j.apenergy.2025.127171

[9] Chen, Y. & Luo, X. (2026). Indoor scan-to-BIM workflows: Progress, challenges, and future directions (2014–2024). *Automation in Construction*, 181, 106578. https://doi.org/10.1016/j.autcon.2025.106578

[10] Mahmoud, M., Li, Y., Adham, M., Mansour, A. & Chen, W. (2026). Indoor 3D point cloud reconstruction for scan-to-BIM automation. *Automation in Construction*, 184, 106853. https://doi.org/10.1016/j.autcon.2026.106853

[11] Xue, T. et al. JaxFEM: A differentiable finite element solver in JAX. https://github.com/tianjuxue/jax-fem

[12] IfcOpenShell contributors. IfcOpenShell — Open Source IFC Library. https://ifcopenshell.org

Tragwerkslabor · Fachbereich Technik · Fachrichtung Bauingenieurwesen · Hochschule Mainz
University of Applied Sciences · philipp.schaefer@hs-mainz.de · April 2026